

Tugas 7: Tugas Praktikum Mandiri 07

Revani – 0110224111 ¹

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: 0110224111@student.nurulfikri.ac.id

Abstract. Pada tugas praktikum mandiri 7 ini, dilakukan penerapan algoritma *Linear Regression* menggunakan dataset satelit yang berisi berbagai fitur *spektral* dan unsur kimia tanah. Tujuan utamanya adalah untuk memprediksi kadar Nitrogen (N) berdasarkan berbagai variabel input seperti P, K, Ca, Mg, Fe, Mn, Cu, Zn, B, serta band-band citra satelit seperti b1–b12 dan nilai-nilai *backscatter* dari radar. Model *regresi linear* ini termasuk jenis *Multiple Linear Regression*, karena menggunakan lebih dari satu variabel independen. Hasil model kemudian dievaluasi menggunakan metrik R^2 (*coefficient of determination*) dan RMSE (*Root Mean Squared Error*) untuk melihat seberapa baik model mampu menjelaskan variasi nilai Nitrogen pada data uji.

1. Penjelasan Hasil Praktikum Mandiri

Pada praktikum ini dilakukan di Google Colab dengan menghubungkan Google Drive agar dataset bisa diakses dengan mudah. Dataset yang digunakan berisi sekitar 30 an kolom dengan data numerik tentang unsur tanah dan band satelit. Setelah dilakukan pembersihan data dan konversi tipe data, model *Linear Regression* dijalankan menggunakan library *scikit-learn*. Model ini kemudian dilatih menggunakan 80% data dan diuji pada 20% data sisanya.

Hasil pelatihan menghasilkan nilai R^2 dan RMSE yang menggambarkan performa model. Jika nilai R^2 mendekati 1 artinya model cukup baik, tapi jika nilai RMSE tinggi maka prediksi model cenderung meleset dari nilai sebenarnya. Dalam konteks tugas ini, model masih menunjukkan selisih tertentu antara nilai aktual dan hasil prediksi, yang artinya masih bisa dikembangkan dengan pemilihan fitur atau metode lain. Berikut ini adalah hasil praktikum yang saya kerjakan pada praktikum mandiri 07.

2. Penjelasan Kode Program

```
from google.colab import drive
drive.mount('/content/drive')
path = "/content/drive/MyDrive/Colab Notebooks/Machine
Learning/Praktikum07"
```

Langkah awal yang dilakukan adalah menghubungkan Google Colab dengan akun Google Drive agar dataset yang tersimpan bisa dibuka. Setelah drive terhubung, digunakan path direktori untuk mengakses file *dataset_satelit.csv* seperti yang ditunjukkan pada Gambar 1.

Menghubungkan Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
path = "/content/drive/MyDrive/Colab Notebooks/Machine Learning/Praktikum07"
```

Gambar 1. Pada bagian ini ditunjukkan menghubungkan *gdrive*.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
```

Selanjutnya mengimpor library seperti pandas untuk pengolahan data, numpy untuk operasi numerik, sklearn untuk membangun model regresi dan menghitung metrik evaluasi seperti yang ditunjukkan pada Gambar 2.

Import Library

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
```

Gambar 2. Pada bagian ini ditunjukkan untuk mengimpor library.

```
df_3 = pd.read_csv(path + "/Data/dataset_satelit.csv")
df_3.head()
```

Bagian ini membaca dataset satelit ke dalam DataFrame `df_3`. Outputnya menampilkan beberapa baris pertama berisi nilai-nilai fitur satelit dan unsur tanah seperti N, P, K, Ca, Mg, Fe, Mn, dan band b1–b12. Interpretasi dataset ini digunakan untuk memprediksi nilai Nitrogen berdasarkan fitur-fitur yang tersedia seperti yang ditunjukkan pada Gambar 3.

Load Data

```
df_3 = pd.read_csv(path + "/Data/dataset_satelit.csv")
df_3.head()
```

	No	Longitude	Latitude	N	P	K	Ca	Mg	Fe	Mn	...	b1	Sigma_VV	Sigma_VH	plia	lia	iafe	gamma0_vv	gamma0_vh	beta0_vv	beta0_vh
0	1	103.036658	-0.604417	2.64	0.15	0.415	0.51	0.31	119.96	463.23	...	0.0433	0.18183	0.04461	35.74446	35.79744	35.41161	0.22331	0.05479	0.31325	0.07686
1	2	103.037201	-0.604689	2.75	0.17	0.568	0.76	0.58	102.63	493.81	...	0.0465	0.22079	0.04640	35.12096	35.14591	35.41510	0.27116	0.05699	0.38033	0.07993
2	3	103.036359	-0.603012	1.77	0.12	0.339	0.49	0.6	107.37	460.93	...	0.0417	0.18926	0.03992	35.07724	35.07730	35.41135	0.23242	0.04902	0.32604	0.06876
3	4	103.036950	-0.603219	2.30	0.15	0.460	0.74	0.67	96.02	338.17	...	0.0367	0.14769	0.03622	36.08078	36.08469	35.41583	0.18138	0.04448	0.25440	0.06238
4	5	103.036802	-0.601969	2.05	0.14	0.308	0.64	0.72	87.01	384.33	...	0.0361	0.18205	0.03797	32.68855	32.69293	35.41592	0.22359	0.04664	0.31359	0.06541

5 rows x 34 columns

Gambar 3. Pada bagian ini ditunjukkan untuk membaca file csv.

```
df_3.info()
df_3.describe()
df_3.columns
```

Tiga baris ini memberikan ringkasan tipe data, jumlah baris dan kolom, serta nilai statistik seperti mean, min, dan max. Ini berguna untuk memastikan data siap digunakan sebelum modeling seperti yang ditunjukkan pada Gambar 4 dan Gambar 5.

```

df_3.info()

... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 594 entries, 0 to 593
Data columns (total 34 columns):
#   Column      Non-Null Count  Dtype
---  -
0   No           594 non-null    int64
1   Longitude    594 non-null    float64
2   Lattitude    594 non-null    float64
3   N            594 non-null    float64
4   P            594 non-null    float64
5   K            593 non-null    float64
6   Ca           594 non-null    float64
7   Mg           594 non-null    object
8   Fe           594 non-null    float64
9   Mn           594 non-null    float64
10  Cu           594 non-null    float64
11  Zn           594 non-null    float64
12  B            594 non-null    float64
13  b12          594 non-null    float64
14  b11          594 non-null    float64
15  b9           594 non-null    float64
16  b8a          594 non-null    float64
17  b8           594 non-null    float64
18  b7           594 non-null    float64
19  b6           594 non-null    float64
20  b5           594 non-null    float64
21  b4           594 non-null    float64
22  b3           594 non-null    float64
23  b2           594 non-null    float64
24  b1           594 non-null    float64
25  Sigma_VV     594 non-null    float64
26  Sigma_VH     594 non-null    float64
27  plia         594 non-null    float64
28  lia          594 non-null    float64
29  iafe         594 non-null    float64
30  gamma0_vv    594 non-null    float64
31  gamma0_vh    594 non-null    float64
32  beta0_vv     594 non-null    float64
33  beta0_vh     594 non-null    float64
dtypes: float64(32), int64(1), object(1)
memory usage: 157.9+ KB

```

Gambar 4. Pada bagian ini ditunjukkan untuk informasi.

df_3.describe()

	No	Longitude	Latitude	N	P	K	Ca	Fe	Mn	Cu	...	bt	Sigma_VV	Sigma_VH	plia	lia	iafe	gamma0_vv	gamma0_vh	beta0_vv	beta0_vh
count	594.000000	594.000000	594.000000	594.000000	594.000000	593.000000	594.000000	594.000000	594.000000	594.000000	...	594.000000	594.000000	594.000000	594.000000	594.000000	594.000000	594.000000	594.000000	594.000000	594.000000
mean	297.500000	106.878644	-1.024933	2.259091	0.141380	0.582175	0.595094	74.613771	308.034697	2.391195	...	0.177291	0.234474	0.102789	28.640422	28.664891	28.609569	0.202587	0.051524	0.269642	0.062320
std	171.617307	4.949840	0.965349	0.395499	0.019782	0.222567	0.366118	55.579655	241.731643	1.580296	...	0.155615	0.070516	0.112310	15.325347	15.380384	15.329170	0.104357	0.012959	0.143728	0.024218
min	1.000000	102.760857	-2.333750	1.140000	0.090000	0.122000	0.050000	21.080000	3.160000	0.090000	...	0.014100	0.115170	0.021460	0.127000	0.098600	0.026000	0.008700	0.016900	0.009300	0.016300
25%	149.250000	102.927811	-2.233338	1.982500	0.130000	0.429000	0.320000	40.795000	124.015000	1.172500	...	0.046925	0.183210	0.039535	31.959745	31.968948	33.685353	0.183085	0.040250	0.244935	0.052772
50%	297.500000	103.581969	-0.602276	2.280000	0.140000	0.549000	0.540000	65.650000	239.445000	2.225000	...	0.072700	0.213385	0.046550	35.067930	35.110415	34.611565	0.233590	0.050415	0.310380	0.068380
75%	445.750000	113.403797	-0.257349	2.570000	0.150000	0.710000	0.790000	87.372500	434.990000	3.357500	...	0.318900	0.262242	0.059190	38.319135	38.441065	39.002760	0.271790	0.060410	0.364505	0.079020
max	594.000000	113.434700	0.069251	3.230000	0.220000	1.489000	2.820000	559.100000	2009.320000	8.170000	...	0.751400	0.512210	0.373000	47.592900	48.014640	39.209330	0.658960	0.122300	0.814170	0.150620

1 rows = 33 columns

df_3.columns

```
Index(['No', 'Longitude', 'Latitude', 'N', 'P', 'K', 'Ca', 'Fe', 'Mn', 'Cu', 'bt', 'Sigma_VV', 'Sigma_VH', 'plia', 'lia', 'iafe', 'gamma0_vv', 'gamma0_vh', 'beta0_vv', 'beta0_vh'], dtype='object')
```

Gambar 5. Pada bagian ini ditunjukkan untuk informasi dataset.

```
df_3['Mg'] = pd.to_numeric(df_3['Mg'], errors='coerce')
df_3 = df_3.dropna()
```

Di sini kolom Mg dikonversi menjadi tipe numerik, dan baris yang memiliki nilai kosong dihapus agar model tidak error saat training seperti yang ditunjukkan pada Gambar 6.

```
#ubah kolom mg menjadi float
df_3['Mg'] = pd.to_numeric(df_3['Mg'], errors='coerce')

#hilangkan baris yang memiliki missing value
df_3 = df_3.dropna()
```

Gambar 6. Pada bagian ini ditunjukkan untuk membersihkan data.

```
X_3 = df_3[['P', 'K', 'Ca', 'Mg', 'Fe', 'Mn', 'Cu', 'Zn', 'B',
            'b12', 'b11', 'b9', 'b8a', 'b8', 'b7', 'b6', 'b5',
            'b4', 'b3', 'b2', 'b1', 'Sigma_VV', 'Sigma_VH', 'plia',
            'lia', 'iafe',
            'gamma0_vv', 'gamma0_vh', 'beta0_vv', 'beta0_vh']]
y_3 = df_3['N']
```

Variabel X_3 berisi semua kolom yang dianggap berpengaruh terhadap Nitrogen, sedangkan y_3 adalah nilai target yaitu N. Karena kolom input lebih dari satu, maka ini termasuk *multiple linear regression* seperti yang ditunjukkan pada Gambar 7.

Feature Selection

```
# Memilih fitur dengan beberapa kolom
# X_3 = df_3[['b2', 'b3', 'b4', 'b8', 'b11', 'Sigma_VV', 'Sigma_VH']]
# y_3 = df_3['N']

# Memilih fitur dengan semua kolom
X_3 = df_3[['P', 'K', 'Ca', 'Mg', 'Fe', 'Mn', 'Cu', 'Zn', 'B',
            'b12', 'b11', 'b9', 'b8a', 'b8', 'b7', 'b6', 'b5',
            'b4', 'b3', 'b2', 'b1', 'Sigma_VV', 'Sigma_VH', 'plia', 'lia', 'iafe',
            'gamma0_vv', 'gamma0_vh', 'beta0_vv', 'beta0_vh']]
y_3 = df_3['N']
```

Gambar 7. Pada bagian ini ditunjukkan feature selection.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_3, y_3, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
```

Data dibagi menjadi 80% untuk pelatihan dan 20% untuk pengujian. Model regresi linear dibuat menggunakan `LinearRegression()` dan dilatih menggunakan data training seperti yang ditunjukkan pada Gambar 8.

Splitting Data

Pembagian data yang menggunakan semua kolom

```
X_train, X_test, y_train, y_test = train_test_split(
    X_3, y_3, test_size=0.2, random_state=42
)
```

```
model = LinearRegression()

#training data
model.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

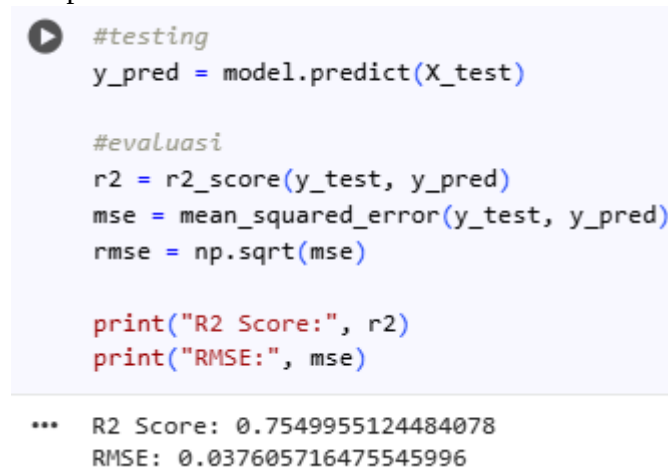
Gambar 8. Pada bagian ini ditunjukkan splitting data dan training model.

```

y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print("R2 Score:", r2)
print("RMSE:", mse)

```

Setelah model dilatih, data uji digunakan untuk melihat kemampuan model dalam memprediksi nilai Nitrogen. R^2 Score menunjukkan seberapa besar variasi data yang bisa dijelaskan oleh model. RMSE menunjukkan rata-rata selisih prediksi terhadap nilai sebenarnya. Jika RMSE tinggi atau R^2 rendah, model dianggap kurang akurat seperti yang ditunjukkan pada Gambar 9.



```

#testing
y_pred = model.predict(X_test)

#evaluasi
r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print("R2 Score:", r2)
print("RMSE:", mse)

```

```

*** R2 Score: 0.7549955124484078
    RMSE: 0.037605716475545996

```

Gambar 9. Pada bagian ini ditunjukkan testing dan evaluasi model.

```

coeff = pd.DataFrame({
    'Fitur': X_3.columns,
    'Koefisien': model.coef_
})

print(coeff)

```

Output ini menampilkan pengaruh setiap fitur terhadap nilai Nitrogen. Nilai koefisien yang besar menunjukkan fitur tersebut punya kontribusi signifikan terhadap hasil prediksi seperti yang ditunjukkan pada Gambar 10.

```
coeff = pd.DataFrame({
    'Fitur': X_3.columns,
    'Koefisien': model.coef_
})

print(coeff)
```

	Fitur	Koefisien
0	P	8.570837
1	K	0.056014
2	Ca	-0.058380
3	Mg	-0.161095
4	Fe	0.000284
5	Mn	0.000177
6	Cu	0.023202
7	Zn	0.002968
8	B	-0.002092
9	b12	0.327312
10	b11	-0.619437
11	b9	-0.047586
12	b8a	0.064453
13	b8	-0.072659
14	b7	0.981186
15	b6	-0.864947
16	b5	-0.329397
17	b4	1.420614
18	b3	-0.449043
19	b2	-0.357454
20	b1	-0.033418
21	Sigma_VV	0.156595
22	Sigma_VH	-1.466200
23	plia	0.021315
24	lia	-0.021906
25	iafe	-0.078122
26	gamma0_vv	0.457327
27	gamma0_vh	2.946416
28	beta0_vv	-0.569281
29	beta0_vh	-0.677599

Gambar 10. Pada bagian ini ditunjukkan untuk menampilkan koefisien regresi.

```
import statsmodels.api as sm
```

```
X_sm = sm.add_constant(X_3)
```

```
model_ols = sm.OLS(y_3, X_sm).fit()
```

```
print(model_ols.summary())
```

Model OLS (Ordinary Least Squares) digunakan untuk analisis statistik lebih detail. Output `summary()` menampilkan nilai *p-value*, *R-squared*, dan uji signifikansi tiap variabel. Nilai *p-*

value yang kecil (< 0.05) menandakan variabel tersebut signifikan mempengaruhi N seperti yang ditunjukkan pada Gambar 11.

```
import statsmodels.api as sm

X_sm = sm.add_constant(X_3)
model_ols = sm.OLS(y_3, X_sm).fit()
print(model_ols.summary())
```

*** OLS Regression Results ***

	coef	std err	t	P> t	[0.025	0.975]
Dep. Variable:			N	R-squared:		0.779
Model:			OLS	Adj. R-squared:		0.768
Method:			Least Squares	F-statistic:		66.18
Date:			Wed, 05 Nov 2025	Prob (F-statistic):		6.69e-163
Time:			14:43:50	Log-Likelihood:		156.68
No. Observations:			592	AIC:		-251.4
Df Residuals:			561	BIC:		-115.5
Df Model:			30			
Covariance Type:			nonrobust			
	coef	std err	t	P> t	[0.025	0.975]
const	3.8681	0.864	4.479	0.000	2.172	5.564
P	8.5760	0.465	18.428	0.000	7.662	9.490
K	0.0273	0.051	0.530	0.596	-0.074	0.128
Ca	-0.0416	0.023	-1.808	0.071	-0.087	0.004
Mg	-0.2166	0.063	-3.464	0.001	-0.339	-0.094
Fe	0.0002	0.000	1.102	0.271	-0.000	0.001
Mn	0.0002	4.59e-05	4.083	0.000	9.72e-05	0.000
Cu	0.0229	0.006	4.046	0.000	0.012	0.034
Zn	0.0024	0.001	1.889	0.059	-9.52e-05	0.005
B	-0.0022	0.001	-1.476	0.141	-0.005	0.001
b12	0.2010	0.534	0.377	0.707	-0.848	1.250
b11	-0.9945	0.517	-1.922	0.055	-2.011	0.022
b9	-0.0802	0.067	-1.191	0.234	-0.212	0.052
b8a	0.0962	0.067	1.427	0.154	-0.036	0.229
b8	-0.0806	0.024	-3.362	0.001	-0.128	-0.034
b7	1.3059	0.542	2.409	0.016	0.241	2.371
b6	-0.9238	0.462	-2.001	0.046	-1.831	-0.017
b5	-0.5470	0.488	-1.120	0.263	-1.506	0.412
b4	1.2561	0.972	1.292	0.197	-0.653	3.166
b3	0.3085	1.639	0.188	0.851	-2.910	3.527
b2	-0.7075	0.942	-0.751	0.453	-2.559	1.144
b1	0.0023	0.352	0.006	0.995	-0.689	0.694
Sigma_VV	0.9543	0.889	1.074	0.283	-0.792	2.700
Sigma_VH	-1.1666	0.861	-1.356	0.176	-2.857	0.524
plia	-0.0113	0.071	-0.159	0.873	-0.150	0.128
lia	0.0102	0.071	0.144	0.886	-0.129	0.149
iafe	-0.0802	0.023	-3.475	0.001	-0.126	-0.035
gamma0_vv	-0.5040	2.100	-0.231	0.817	-4.785	3.777
gamma0_vh	3.1827	4.643	0.685	0.493	-5.938	12.303
beta0_vv	-0.2381	1.580	-0.151	0.880	-3.342	2.865
beta0_vh	-0.5472	3.536	-0.155	0.877	-7.493	6.399
Omnibus:		28.388	Durbin-Watson:			1.527
Prob(Omnibus):		0.000	Jarque-Bera (JB):			53.216
Skew:		-0.310	Prob(JB):			2.78e-12
Kurtosis:		4.332	Cond. No.			2.98e+05

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 2.98e+05. This might indicate that there are strong multicollinearity or other numerical problems.

Gambar 11. Pada bagian ini ditunjukkan model OLS.

```
import matplotlib.pyplot as plt

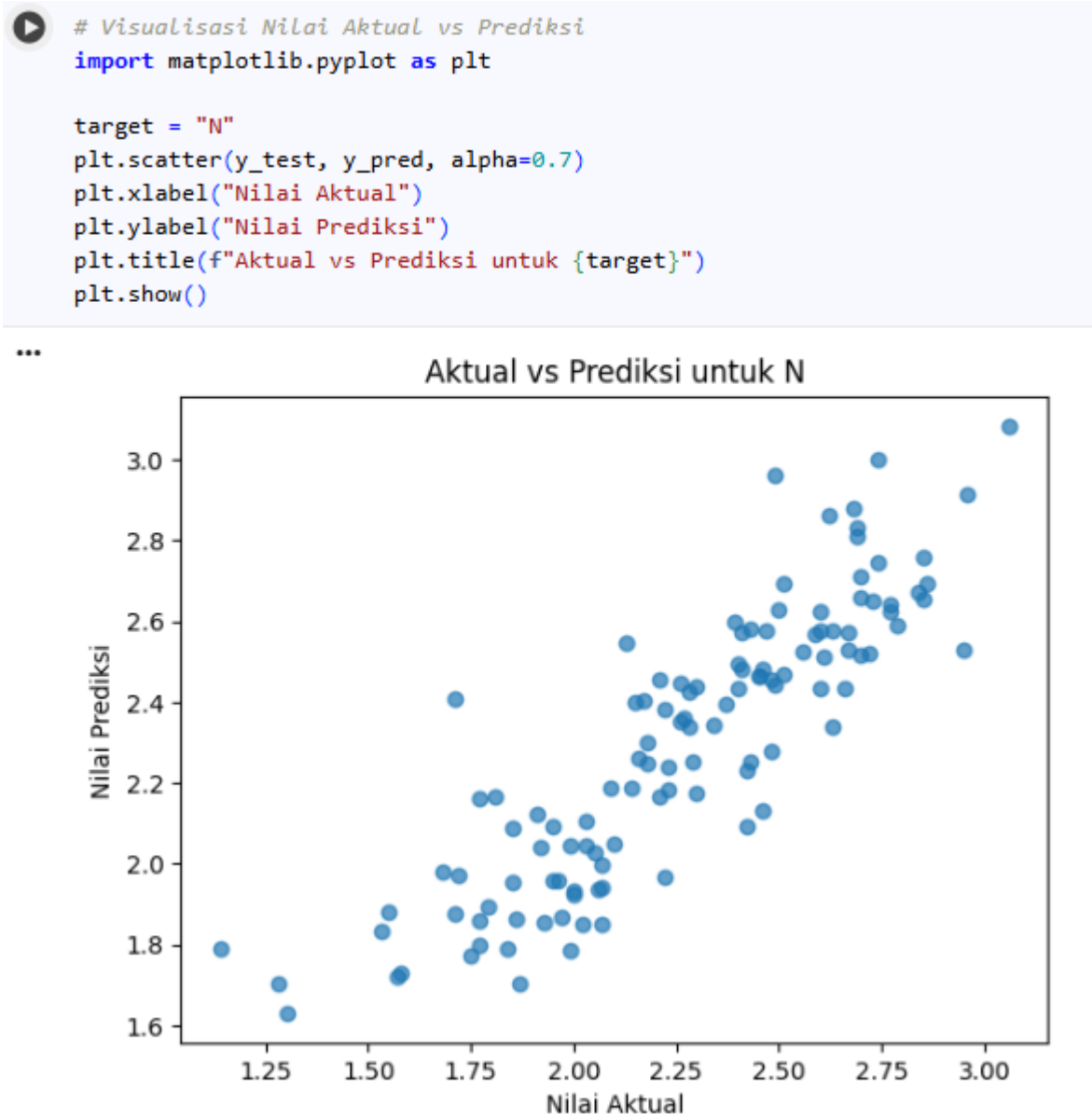
target = "N"

plt.scatter(y_test, y_pred, alpha=0.7)

plt.xlabel("Nilai Aktual")
```

```
plt.ylabel("Nilai Prediksi")
plt.title(f"Aktual vs Prediksi untuk {target}")
plt.show()
```

Grafik ini memperlihatkan sebaran antara nilai aktual (data uji) dengan hasil prediksi model. Semakin mendekati garis diagonal ($y = x$), berarti model semakin akurat. Jika titik-titik tersebar jauh dari garis, berarti prediksi masih kurang tepat seperti yang ditunjukkan pada Gambar 12.



Gambar 12. Pada bagian ini ditunjukkan visualisasi nilai aktual vs prediksi.

3. Kesimpulan dan Hasil Implementasi

Dari hasil implementasi *Multiple Linear Regression* pada dataset satelit, didapatkan model yang mampu memprediksi kadar Nitrogen berdasarkan berbagai fitur spektral dan unsur tanah. Nilai R^2 menunjukkan seberapa besar hubungan linear yang berhasil ditangkap oleh model, sedangkan RMSE menunjukkan tingkat kesalahan prediksi.

Meskipun model sudah berjalan dengan baik, masih terdapat selisih antara nilai aktual dan prediksi, yang artinya model belum sepenuhnya akurat. Hal ini bisa disebabkan oleh adanya hubungan non-linear antar variabel atau beberapa fitur yang kurang relevan.

Secara keseluruhan, tugas ini memberikan pemahaman bagaimana algoritma regresi linear bekerja untuk menganalisis hubungan antar variabel pada data satelit, serta bagaimana cara mengevaluasi dan memvisualisasikan hasil prediksi.

4. Link GitHub (Praktikum dan Tugas Mandiri 07)

https://github.com/revani18/ML_2025_Revani_3AI01/tree/def505ff7a2f8522ac0fb3be8b73dbedbc510779/Praktikum07